作品名　スケルトンラッシュ

ジャンル　3Dアクションゲーム

開発環境　C++,DxLib

対応機種　windows

制作期間　2か月

制作時期　2026年3月～

制作人数　1人



# ゲーム概要

ウェーブごとに敵が出現する




逃げ回る敵をすべて倒したらクリア



# 技術紹介①
# ヒットストップ&カメラシェイク

攻撃が当たった際にエフェクト以外で派手さを演出したい

⇩

ヒットストップとカメラシェイクをそれぞれ実装

⇩

演出が強化され、調整もしやすくなった

```
void HitStopManager::Update()
{
    m_delayFrame--;
    if (m_delayFrame < 0)
    {
        if (m_hitStopFrame > 0)
        {
            m_hitStopFrame--;
        }
    }
}

void HitStopManager::Start(int frame, int delay)
{
    m_delayFrame = delay;

    if (frame > m_hitStopFrame)
    {
        m_hitStopFrame = frame;
    }
}
```

```
void Camera::ShakeStart(float power, int time,int delay)
{
    m_shakePower = power;
    m_shakeTime = time;
    m_shakeDilayTime = delay;
}

void Camera::ShakeUpdate()
{
    m_shakeOffset = { 0.0f,0.0f,0.0f };
    if(m_shakeDilayTime < 0)
    {
        if (m_shakeTime > 0)
        {
            m_shakeTime--;
            m_shakeOffset.x = (rand() / (float)RAND_MAX - 0.5f) * 2.0f * m_shakePower;
            m_shakeOffset.y = (rand() / (float)RAND_MAX - 0.5f) * 2.0f * m_shakePower;
            m_shakeOffset.z = (rand() / (float)RAND_MAX - 0.5f) * 2.0f * m_shakePower;
        }
    }
    m_shakeDilayTime--;
}
```

# 技術紹介②
# アニメーション
# マネージャー

アニメーション機能を一つのクラス
にまとめたい

⇩

アニメーションを名前で管理して
現在と前回のアニメーションを保存する

⇩

同じアニメーションの再生を防止
ブレンド処理を入れることにより
自然なアニメーション遷移ができるように

```
//ブレンド処理
if (m_prevAnim >= 0)
{
    //前のアニメーションの時間更新
    m_prevTime += 0.5f;
    MV1SetAttachAnimTime(m_modelHandle, m_prevAttachNo, m_prevTime);
    //ブレンド時間の更新
    m_blendTimer += 1.0f;
    //ブレンドレート
    float rate = 1.0f;

    if (m_blendTime > 0.0f)
    {
        rate = m_blendTimer / m_blendTime;
    }

    if (rate > 1.0f)
    {
        rate = 1.0f;
    }
    //新しいアニメーション
    MV1SetAttachAnimBlendRate(m_modelHandle, m_currentAttachNo, rate);

    //前のアニメーション
    MV1SetAttachAnimBlendRate(m_modelHandle, m_prevAttachNo, 1.0f - rate);
```

```
void AnimationController::Play(const std::string& name, bool loop, float blendTim
{
    //アニメーション名で探す
    auto it = m_animTable.find(name);
    if (it == m_animTable.end())
    {
        return;
    }

    int newAnim = it->second;
    //同じアニメーションなら再生しない
    if (newAnim == m_currentAnim)
    {
        return;
    }

    //現在のアニメーションを前のアニメーションとして保存
    MV1DetachAnim(m_modelHandle, m_prevAttachNo);
    m_prevAnim = m_currentAnim;
    m_prevAttachNo = m_currentAttachNo;
    m_prevTime = m_currentTime;

    //新しいアニメーションを設定
    m_currentAnim = newAnim;
    m_currentTime = 0.0f;
    m_isLoop = loop;

    m_currentAttachNo = MV1AttachAnim(m_modelHandle, m_currentAnim);
```